

Deux manières de définir des agents

Dans les modules 1 à 3, vous avez créé des agents à l'aide de **code Python** dans `agent.py` :

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Aide les élèves à apprendre l'algèbre',
    instruction='Tu es un professeur de mathématiques patient...'
)
```

C'est idéal pour le développement. Mais ADK permet également de définir des agents à l'aide de **fichiers de configuration YAML**, ce qui peut faciliter :

- la modification par des non-programmeurs ;
- le contrôle des versions et le partage ;
- l'exécution de tests rapides ;
- la séparation de la configuration et du code.

Ce module présente la **configuration d'agent**, l'approche d'ADK basée sur YAML pour créer des agents.

Remarque concernant les langages compatibles

Important : ADK accepte plusieurs langages de programmation :

- **Python** : totalement compatible avec le code Python et la configuration d'agent YAML.
- **Java** : accepte les agents basés sur du code Python (consultez le [guide de démarrage rapide Java](#)).

La configuration d'agent (YAML) est actuellement uniquement compatible avec Python.

Extrait de la documentation ADK :

"Langage de programmation : la fonctionnalité de configuration d'agent n'accepte que le code Python pour les outils et les autres fonctionnalités nécessitant du code de programmation."

Ce module se concentre sur la fonctionnalité de configuration d'agent YAML, disponible lorsque vous utilisez ADK avec Python.

Deux méthodes pour définir des agents

Référence : [Documentation ADK – Configuration d'agent](#)

Méthode 1 : Code Python (modules 1 à 3)

Créé avec : `adk create my_agent`

Génère :

```
my_agent/
├── agent.py      # Code Python définissant votre agent
├── __init__.py
└── .env
```

agent.py :

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='assistant_agent',
    description='Un assistant serviable',
    instruction='Tu es un assistant serviable.'
)
```

Méthode 2 : Configuration YAML (ce module)

Créé avec : `adk create --type=config my_agent`

Génère :

```
my_agent/
├── root_agent.yaml # Fichier de configuration YAML
└── .env
```

root_agent.yaml :

```
name: assistant_agent
model: gemini-2.5-flash
description: Un assistant serviable
instruction: Tu es un assistant serviable.
```

Extrait de la documentation ADK :

"La fonctionnalité de configuration d'agent d'ADK vous permet de créer un workflow ADK sans écrire de code. Une configuration d'agent utilise un fichier texte au format YAML avec une brève description de l'agent."

Créer un agent basé sur YAML

Référence : [Documentation ADK – Configuration d'agent : créer un agent](#)

Nous allons maintenant créer votre premier agent basé sur YAML, étape par étape. Le processus est semblable à celui que vous avez découvert dans le module 1, avec toutefois une différence essentielle : au lieu de générer du code Python, nous allons générer un fichier de

configuration YAML.

Étape 1 : Créer le projet d'agent

Exécutez la commande suivante pour créer un agent basé sur une configuration :

```
adk create --type=config my_config_agent
```

Notez le flag `--type=config`. Il indique à ADK de créer un agent basé sur YAML au lieu de l'agent par défaut basé sur du code Python.

Extrait de la documentation ADK :

"Cette commande génère un dossier `my_agent/` contenant un fichier `root_agent.yaml` et un fichier `.env`."

Un nouveau dossier `my_config_agent/` contenant les éléments suivants s'affiche :

- `root_agent.yaml` : configuration de votre agent (au lieu de `agent.py`)
- `.env` : variables d'environnement pour les clés API (comme dans le module 1)

Étape 2 : Configurer votre clé API

Comme dans le module 1, vous devez configurer votre fichier `.env` avec votre clé API. Le processus est identique : ouvrez `my_config_agent/.env` et ajoutez votre `GOOGLE_API_KEY` comme vous l'avez fait à l'étape 7 du module 1.

Si vous avez besoin d'une piqure de rappel, reportez-vous au [Module 1 – Étape 7 : Configurer votre clé API](#).

Étape 3 : Modifier la configuration YAML

C'est là que les choses deviennent intéressantes. Au lieu d'écrire du code Python, vous allez modifier un fichier YAML.

Ouvrez `root_agent.yaml` dans votre éditeur de texte. Le fichier généré par défaut se présente comme suit :

```
# yaml-language-server:
$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google
/adk/agents/config_schemas/AgentConfig.json
name: assistant_agent
model: gemini-2.5-flash
description: Un agent d'assistance qui peut répondre aux questions des utilisateurs.
instruction: Tu es un agent chargé d'aider les utilisateurs en répondant à leurs
questions.
```

Il s'agit d'une configuration d'agent de base avec quatre paramètres clés. Vous pouvez constater que YAML est plus clair et plus lisible que le code Python. Il n'y a pas d'importations ni de définitions de fonction, mais simplement des paires clé-valeur.

Personnalisons-la pour créer le même agent professeur de mathématiques que celui que vous avez créé dans le module 2. Modifiez votre fichier `root_agent.yaml` pour qu'il ressemble à ceci :

```
# yaml-language-server:
$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json
name: math_tutor_agent
model: gemini-2.5-flash
description: Aide les élèves à apprendre l'algèbre en les guidant à travers les étapes de résolution de problèmes.
instruction: |
  Tu es un professeur d'algèbre patient et encourageant.

  Ton approche pédagogique :
  1. Lorsqu'un élève pose une question, commence par identifier ce qui le met en difficulté
  2. Décompose le problème en étapes plus petites et gérables
  3. Guide l'élève pour qu'il découvre la réponse plutôt que de la lui donner directement
  4. Encourage positivement les efforts et les progrès de l'élève
  5. Utilise un langage simple et évite le jargon

  Garde toujours un ton bienveillant et patient. L'apprentissage prend du temps, et chaque question est une opportunité de progresser.
```

Comprendre la syntaxe YAML :

La barre verticale `|` après `instruction:` indique à YAML que tout ce qui suit est du texte multiligne. C'est idéal pour rédiger des instructions détaillées qui s'étendent sur plusieurs lignes. En général, vous n'avez pas besoin de guillemets ni d'échappement spécial. Il vous suffit de rédiger vos instructions naturellement.

Chaque paramètre correspond directement à ce que vous avez appris dans le module 2 :

- `name` : identifiant unique de l'agent
- `model` : LLM à utiliser (`gemini-2.5-flash`)
- `description` : ce que fait l'agent (pour que les autres agents comprennent)
- `instruction` : comportement attendu de l'agent (partie la plus importante)

Étape 4 : Exécuter votre agent basé sur YAML

Bonne nouvelle : **l'exécution d'un agent basé sur YAML est exactement identique à celle d'un agent basé sur Python.** Toutes les commandes que vous avez apprises dans le module 3 fonctionnent de la même manière.

Depuis votre répertoire d'espace de travail (le dossier parent contenant `my_config_agent`), exécutez la commande suivante :

```
adk web my_config_agent
```

Le fonctionnement est le même que dans les modules 1 et 2. ADK charge votre fichier `root_agent.yaml`, crée l'agent en fonction de cette configuration et démarre l'interface Web.

Extrait de la documentation ADK :

"Vous pouvez exécuter votre agent défini avec la fonctionnalité de configuration d'agent... `adk web` : exécute l'interface utilisateur Web de votre agent."

Une fois l'interface Web ouverte, interagissez avec votre agent professeur de mathématiques comme vous l'avez fait dans le module 2. Essayez de demander : "Comment résoudre l'équation $2x + 5 = 13$?" et laissez-vous guider pas à pas.

YAML ou Python : Quand utiliser chaque langage

Maintenant que vous avez découvert les deux approches, vous vous demandez peut-être quand utiliser la configuration YAML plutôt que le code Python. Les deux méthodes permettent de créer des agents qui fonctionnent de la même manière, mais elles sont adaptées à des scénarios différents.

Aspect	Configuration YAML	Code Python
Facilité de modification	✅ Très facile (fichier texte)	⚠️ Nécessite des connaissances en codage
Application idéale	Agents simples, prototypes rapides	Agents complexes, logique personnalisée
Contrôle des versions	✅ Différences claires	⚠️ Modifications de code
Flexibilité	⚠️ Limitée aux options de configuration	✅ Contrôle programmatique total
Non-programmeurs	Peuvent effectuer des modifications	❌ Doivent avoir des compétences en codage
Fonctionnalités avancées	⚠️ Bientôt disponible	✅ Accès total

Choisissez YAML dans les cas suivants :

Vous recherchez de la simplicité et de l'accessibilité. La configuration YAML est parfaite si vous créez des agents simples et ne voulez pas vous compliquer la tâche. Les non-programmeurs peuvent facilement modifier un fichier YAML pour ajuster le comportement de l'agent sans toucher au code. C'est idéal pour la collaboration entre des équipes dont tous

les membres n'ont pas d'expérience en codage.

Vous avez besoin d'exécuter des tests rapides. Vous souhaitez tester différentes instructions ou passer d'un modèle à un autre ? Il vous suffit de modifier le fichier YAML et de redémarrer votre agent. Vous n'avez pas besoin de parcourir les importations Python et les définitions de classe.

La configuration doit être séparée du code. YAML permet de séparer clairement la configuration de l'agent de toute logique personnalisée que vous pourriez ajouter ultérieurement. Cela facilite la gestion des différentes configurations pour les environnements de développement, de préproduction et de production.

Choisissez Python dans les cas suivants :

Vous créez des systèmes multi-agents complexes. Comme vous l'apprendrez dans le cours 7, les architectures multi-agents sophistiquées avec délégation, agents de workflow et orchestration personnalisée nécessitent toute la puissance du code Python. YAML est limité aux options de configuration de base.

Vous avez besoin d'outils et de rappels personnalisés. Dans les cours 4 et 5, vous apprendrez à ajouter des outils personnalisés et à implémenter des rappels. Ces fonctionnalités avancées nécessitent du code Python et ne peuvent pas être entièrement exprimées en YAML seul.

Vous souhaitez un contrôle programmatique. Il arrive que vous deviez générer dynamiquement des configurations d'agents, modifier le comportement en fonction de conditions d'exécution ou intégrer étroitement votre agent à d'autres bibliothèques Python. Le code Python vous offre cette flexibilité.

Exemple : Comparaison côte à côte

Pour bien comprendre l'équivalence entre YAML et Python, examinons le même agent simple défini avec les deux approches. Cet agent d'accueil comporte les quatre mêmes paramètres que ceux que nous avons utilisés tout au long de ce cours.

Code Python (agent.py) :

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='greeting_agent',
    description='Un agent d'accueil serviable',
    instruction='Accueille les utilisateurs de manière chaleureuse et professionnelle.'
```

)

Configuration YAML (root_agent.yaml) :

```
name: greeting_agent
model: gemini-2.5-flash
description: Un agent accueillant et chaleureux
instruction: Accueille les utilisateurs de manière chaleureuse et professionnelle.
```

Résultat : les deux approches produisent exactement le même comportement d'agent. Lorsque vous exécutez l'une ou l'autre version avec `adk web`, vous obtenez un agent d'accueil identique. La version YAML est juste plus claire et plus facile à lire : pas d'importations, pas de parenthèses, pas de guillemets (dans la plupart des cas), juste une configuration simple.

Ce que vous avez accompli

Félicitations ! Vous avez appris qu'il existe **deux manières de définir des agents dans ADK**, et que les deux sont tout aussi valables selon vos besoins.

Vous avez commencé par écrire du code Python dans les modules 1 à 3, où vous avez créé des agents en écrivant des fichiers `agent.py` avec des importations et une instanciation de classe. Vous avez ainsi pu vous familiariser avec la structure de base des agents.

Vous avez ensuite découvert la configuration YAML, une méthode plus simple et sans code pour définir des agents. Vous avez appris que `adk create --type=config` génère un fichier `root_agent.yaml` au lieu de code Python, ce qui permet aux non-programmeurs de créer des agents.

Surtout, vous avez vu que les deux approches produisent des agents identiques. Que vous définissiez votre professeur de mathématiques en Python ou en YAML, son comportement sera exactement le même. Les commandes `adk web`, `adk run` et `adk api_server` fonctionnent pour les deux approches.

Vous avez également appris à choisir la méthode la plus adaptée : YAML pour la simplicité et le prototypage rapide, et Python pour les systèmes multi-agents complexes et les fonctionnalités avancées que vous découvrirez dans les prochains cours.

Enfin, vous savez que **la fonctionnalité de configuration d'agent est actuellement uniquement compatible avec Python**, même si ADK lui-même accepte plusieurs langages de programmation, y compris Java. À mesure que le framework évolue, la prise en charge de YAML peut s'étendre à d'autres langages.

Points à retenir

ADK vous offre une grande flexibilité dans la définition des agents. Vous n'avez pas besoin d'écrire du code si vous ne le souhaitez pas. La configuration YAML offre une alternative claire et accessible pour les agents simples.

La **configuration YAML est idéale** lorsque vous avez besoin d'effectuer des tests rapides, lorsque des membres de l'équipe n'ayant pas un profil technique doivent ajuster le comportement de l'agent ou lorsque vous souhaitez séparer clairement la configuration et le code. Vous pouvez modifier un fichier YAML, l'enregistrer et redémarrer votre agent sans aucune programmation.

Le code Python devient essentiel à mesure que vos agents gagnent en sophistication. Lorsque vous commencerez à ajouter des outils personnalisés (cours 4), à implémenter des rappels (cours 5) ou à créer des systèmes multi-agents (cours 7), vous aurez besoin de toute la puissance de Python. Mais vous pouvez toujours commencer par YAML et passer à Python ultérieurement.

Les deux méthodes sont des objets de première classe dans ADK. Il n'y a pas de bon ou de mauvais choix. Choisissez l'approche qui correspond le mieux à vos besoins actuels. N'oubliez pas que l'équipe ADK développe activement la fonctionnalité de configuration d'agent et qu'elle apprécie les commentaires.

Extrait de la documentation ADK :

"La fonctionnalité de configuration d'agent est expérimentale et présente certaines limites connues. N'hésitez pas à nous transmettre vos commentaires !"